Level 5: Subqueries

A subquery is a query nested inside another one. This level covers the full toolkit: simple aggregate subqueries, correlated subqueries (which re-run per outer row), EXISTS/NOT EXISTS, IN/NOT IN, ANY/ALL, and the classic 'Nth highest value' and 'find the argmax group' patterns that come up constantly in interviews.

Q111. Second highest salary MEDIUM

```
SELECT MAX(salary) FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

WHAT THIS QUERY DOES

Identical to Q25 — included again here as this level's opening example of the nested-subquery technique, since the whole rest of Level 5 builds on being comfortable with a query inside a query.

RESULT ON OUR SAMPLE DATA

MAX(salary)
91,000

1 row returned.

Q112. Third highest salary MEDIUM

```
SELECT MAX(salary) FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees
                WHERE salary < (SELECT MAX(salary) FROM employees));
```

WHAT THIS QUERY DOES

Nesting the same trick from Q111 one level deeper: the innermost subquery finds the highest salary, the middle layer finds the highest salary *below* that (the 2nd highest), and the outer query finds the highest salary below *that* (the 3rd highest). Each layer peels off one more 'top' value — but as you can see, this gets unwieldy fast, which is exactly why Q113 introduces a cleaner, more general approach for 'Nth highest' that doesn't require manually nesting a nother layer for every N.

RESULT ON OUR SAMPLE DATA

MAX(salary)
90,000

1 row returned.

Q113. Nth highest salary (generic) HARD

```
-- For N=4
SELECT DISTINCT salary
FROM employees e1
WHERE 4 = (SELECT COUNT(DISTINCT salary)
           FROM employees e2
           WHERE e2.salary >= e1.salary);
```

WHAT THIS QUERY DOES

A cleaner, reusable alternative to the nested-MAX approach: for each distinct salary in the table, count how many *distinct* salaries are greater than or equal to it. A salary that's the highest will have exactly 1 salary ≥ itself (only itself); the 2nd highest will have exactly 2 (itself and the highest); and so on. So 'find the salary where this count equals 4' directly gives you the 4th-highest salary, no matter how large N is — just change the number, without adding another layer of nesting. This correlated subquery pattern is a genuinely important one to memorize for interviews, since 'Nth highest X' is an extremely common question.

RESULT ON OUR SAMPLE DATA

salary
88,000

1 row returned.

Q114. Salary above average EASY

```
SELECT name, salary FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

WHAT THIS QUERY DOES

Identical to Q38 — the single-value subquery pattern (compute one number, then filter rows against it) shown again here as this level's canonical simple example.

RESULT ON OUR SAMPLE DATA

name	salary
Rahul Singh	85,000
Vikram Reddy	90,000
Karthik Nair	95,000
Arjun Reddy	82,000
Suresh Babu	88,000
Ravi Teja	91,000

6 rows returned.

Q115. Salary below average EASY

```
SELECT name, salary FROM employees
WHERE salary < (SELECT AVG(salary) FROM employees);
```

WHAT THIS QUERY DOES

The mirror image of Q114 — same subquery, opposite comparison direction, to find who's earning below the company-wide average instead of above it.

RESULT ON OUR SAMPLE DATA

name	salary
Aarav Sharma	75,000
Priya Patel	65,000
Neha Gupta	72,000
Ananya Menon	60,000
Meera Krishnan	68,000
Sneha Patel	58,000
Rajesh Kumar	70,000
Deepa Iyer	64,000
Kavya Nair	67,000

9 rows returned.

Q116. Department average comparison MEDIUM

```
SELECT name, salary, department
FROM employees e
WHERE salary > (SELECT AVG(salary) FROM employees WHERE department =
e.department);
```

WHAT THIS QUERY DOES

Identical to Q123 (they're the same query, included twice by the original cheat sheet) — this is a **correlated subquery**: the inner query's `WHERE department = e.department` reaches out to the outer query's current row, so the 'average' being compared against is recalculated fresh for each employee's own department, not one single company-wide number. See Q123 for the fuller explanation of how correlated subqueries differ from the simple kind in Q114/115.

RESULT ON OUR SAMPLE DATA

name	salary	department
Priya Patel	65,000	HR
Rahul Singh	85,000	Finance
Vikram Reddy	90,000	IT
Karthik Nair	95,000	IT
Meera Krishnan	68,000	Marketing
Deepa Iyer	64,000	HR
Suresh Babu	88,000	IT
Ravi Teja	91,000	IT

8 rows returned.

Q117. EXISTS - Employees with orders MEDIUM

```
SELECT name FROM employees e
WHERE EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = e.emp_id);
```

WHAT THIS QUERY DOES

EXISTS is a true/false check: it doesn't care *what* the subquery returns (notice it just selects the literal 1, a throwaway placeholder), only *whether it returns any rows at all* for the current outer row. This keeps an employee only if at least one order exists with a matching `customer_id`. In our sample schema, `orders.customer_id` actually refers to `customers`, not `employees` — so this query is really testing 'does this employee's ID number happen to coincidentally match some customer's order,' a coincidental/illustrative overlap in the ID numbering rather than a real business relationship, which is worth noticing since it's a good reminder to always double check what a foreign-key-like column is actually supposed to reference.

RESULT ON OUR SAMPLE DATA

name
Aarav Sharma
Priya Patel
Rahul Singh
Neha Gupta
Vikram Reddy
Ananya Menon
Karthik Nair
Meera Krishnan
Arjun Reddy
Sneha Patel

10 rows returned.

Q118. NOT EXISTS - Employees without orders MEDIUM

```
SELECT name FROM employees e
WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = e.emp_id);
```

WHAT THIS QUERY DOES

The negation of Q117 — keeps an employee only if the subquery finds *zero* matching rows. EXISTS/NOT EXISTS are often the most efficient way to write these existence-checks, since the database can stop searching the instant it finds (or confirms the absence of) a single matching row, without needing to count or collect every match the way IN sometimes implies.

RESULT ON OUR SAMPLE DATA

name
Rajesh Kumar
Deepa Iyer
Suresh Babu
Kavya Nair
Ravi Teja

5 rows returned.

Q119. IN - Employees in IT MEDIUM

```
SELECT name FROM employees
WHERE department IN (SELECT dept_name FROM departments WHERE location =
'Mumbai');
```

WHAT THIS QUERY DOES

IN checks whether a value matches *any* value in a list — and that list can come from a subquery instead of being typed out by hand. Here, the inner query first finds every department located in Mumbai, and the outer query then keeps employees whose department name appears anywhere in that resulting list. Despite the question's title mentioning 'IT,' the actual query filters by *location*, not department name — IT just happens to be one of the departments based in Mumbai in our sample data.

RESULT ON OUR SAMPLE DATA

name
Aarav Sharma
Rahul Singh
Neha Gupta

name
Vikram Reddy
Karthik Nair
Arjun Reddy
Rajesh Kumar
Suresh Babu
Kavya Nair
Ravi Teja

10 rows returned.

Q120. NOT IN - Employees not in IT MEDIUM

```
SELECT name FROM employees
WHERE department NOT IN (SELECT dept_name FROM departments WHERE location =
'Mumbai');
```

WHAT THIS QUERY DOES

The negation of Q119 — keeps employees whose department is *not* among the Mumbai-based ones. One important gotcha worth knowing for interviews: `NOT IN` behaves unexpectedly if the subquery's result list contains a `NULL` — it can cause the whole query to return zero rows unexpectedly. `NOT EXISTS` doesn't have this problem, which is why many experienced SQL developers prefer `NOT EXISTS` over `NOT IN` as a matter of habit.

RESULT ON OUR SAMPLE DATA

name
Priya Patel
Ananya Menon
Meera Krishnan
Sneha Patel
Deepa Iyer

5 rows returned.

Q121. ANY - Salary greater than any IT employee HARD

```
SELECT name, salary FROM employees
WHERE salary > ANY (SELECT salary FROM employees WHERE department = 'IT');
```

WHAT THIS QUERY DOES

> ANY means 'greater than *at least one* value in the list' — which is logically the same as being greater than the *smallest* value in that list. So this keeps anyone earning more than the lowest-paid IT employee. *Note: SQLite doesn't support the ANY/ALL keywords directly (they're a MySQL/PostgreSQL/SQL Server feature), so the live example below runs the logically equivalent `salary > (SELECT MIN(salary) FROM employees WHERE department = 'IT')` — same meaning, different syntax.*

RESULT ON OUR SAMPLE DATA

name	salary
Aarav Sharma	75,000
Rahul Singh	85,000
Vikram Reddy	90,000
Karthik Nair	95,000
Arjun Reddy	82,000
Suresh Babu	88,000
Ravi Teja	91,000

7 rows returned.

Q122. ALL - Salary greater than all Marketing employees HARD

```
SELECT name, salary FROM employees
WHERE salary > ALL (SELECT salary FROM employees WHERE department = 'Marketing');
```

WHAT THIS QUERY DOES

> ALL means 'greater than *every single* value in the list' — equivalent to being greater than the *largest* value in that list. This keeps only employees who out-earn even the highest-paid Marketing employee. *(Again executed here as `salary > (SELECT MAX(salary) FROM employees WHERE department = 'Marketing')` since SQLite has no native ALL keyword.)* A useful memory trick: ANY is the 'easy to satisfy' one (beat just one value = beat the minimum), while ALL is the 'hard to satisfy' one (beat every value = beat the maximum).

RESULT ON OUR SAMPLE DATA

name	salary
Aarav Sharma	75,000
Rahul Singh	85,000
Neha Gupta	72,000
Vikram Reddy	90,000
Karthik Nair	95,000
Arjun Reddy	82,000
Rajesh Kumar	70,000

name	salary
Suresh Babu	88,000
Ravi Teja	91,000

9 rows returned.

Q123. Correlated subquery - Salary above department average

MEDIUM

```
SELECT name, salary, department
FROM employees e1
WHERE salary > (SELECT AVG(salary) FROM employees e2 WHERE e1.department =
e2.department);
```

WHAT THIS QUERY DOES

This is the fullest explanation of the correlated subquery pattern seen a few times already (Q62/63, Q116): unlike Q114's subquery (which computes one single number, totally independent of the outer query), this inner query's condition `e1.department = e2.department` reaches back out and references the *current outer row* — meaning conceptually, the subquery has to be re-evaluated separately for every single employee, each time computing 'the average salary within *this employee's own* department' rather than one company-wide figure. It's the single most powerful subquery pattern in this book precisely because it lets each row be judged against a standard computed just for its own group.

RESULT ON OUR SAMPLE DATA

name	salary	department
Priya Patel	65,000	HR
Rahul Singh	85,000	Finance
Vikram Reddy	90,000	IT
Karthik Nair	95,000	IT
Meera Krishnan	68,000	Marketing
Deepa Iyer	64,000	HR
Suresh Babu	88,000	IT
Ravi Teja	91,000	IT

8 rows returned.

Q124. Employees earning more than manager MEDIUM

```
SELECT e.name AS employee, e.salary AS emp_salary,
       m.name AS manager, m.salary AS mgr_salary
```

```
FROM employees e
JOIN employees m ON e.manager_id = m.emp_id
WHERE e.salary > m.salary;
```

WHAT THIS QUERY DOES

No subquery here at all — this reuses the employee-manager self join from Q80/108, then simply filters with a plain `WHERE` comparing two columns from the joined result. It's included in the Subqueries level as a useful contrast: some 'compare against a related value' questions are better solved with a JOIN than a subquery, and recognizing which tool fits is as important as knowing the syntax for both.

RESULT ON OUR SAMPLE DATA

employee	emp_salary	manager	mgr_salary
Rahul Singh	85,000	Aarav Sharma	75,000
Vikram Reddy	90,000	Aarav Sharma	75,000
Karthik Nair	95,000	Aarav Sharma	75,000
Ravi Teja	91,000	Aarav Sharma	75,000

4 rows returned.

Q125. Products never ordered MEDIUM

```
SELECT product_name FROM products
WHERE product_id NOT IN (SELECT DISTINCT product_id FROM order_items);
```

WHAT THIS QUERY DOES

The same underlying question as Q85, now solved with `NOT IN` instead of a LEFT JOIN + NULL check — a good demonstration that many 'find unmatched rows' problems can be written either way. The subquery collects every product ID that appears anywhere in `order_items`, and the outer query keeps products whose ID isn't in that list.

RESULT ON OUR SAMPLE DATA

product_name
Webcam HD
Printer
USB Hub

3 rows returned.

Q126. Customers ordering all products HARD

```
SELECT c.name
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
JOIN order_items oi ON o.order_id = oi.order_id
GROUP BY c.customer_id, c.name
HAVING COUNT(DISTINCT oi.product_id) = (SELECT COUNT(*) FROM products);
```

WHAT THIS QUERY DOES

A 'relational division' style query: join everything together, group by customer, and count how many *distinct* products each customer has ever ordered — then use `HAVING` to keep only customers whose distinct-product count matches the *total* number of products that exist (computed by a subquery). In plain English: 'find customers who have, between all their orders, bought one of literally everything.' With 10 products in our catalog and no single customer having ordered that broadly, this correctly returns 0 rows here, but the technique itself is a classic interview pattern worth knowing.

RESULT ON OUR SAMPLE DATA

name
(no rows returned)

0 rows returned.

Q127. Department maximum salary with employee name MEDIUM

```
SELECT name, department, salary
FROM employees e
WHERE salary = (SELECT MAX(salary) FROM employees WHERE department =
e.department);
```

WHAT THIS QUERY DOES

Identical to Q62 — the correlated-subquery technique for finding the top earner's full row in every group, shown again here as this level's canonical 'match against a per-group MAX' example.

RESULT ON OUR SAMPLE DATA

name	department	salary
Priya Patel	HR	65,000
Rahul Singh	Finance	85,000
Karthik Nair	IT	95,000
Meera Krishnan	Marketing	68,000

4 rows returned.

Q128. Department minimum salary with employee name MEDIUM

```
SELECT name, department, salary
FROM employees e
WHERE salary = (SELECT MIN(salary) FROM employees WHERE department =
e.department);
```

WHAT THIS QUERY DOES

Identical to Q63 — the MIN version of Q127's pattern.

RESULT ON OUR SAMPLE DATA

name	department	salary
Neha Gupta	IT	72,000
Ananya Menon	HR	60,000
Sneha Patel	Marketing	58,000
Kavya Nair	Finance	67,000

4 rows returned.

Q129. Latest order for each customer MEDIUM

```
SELECT customer_id, order_date, amount
FROM orders o1
WHERE order_date = (SELECT MAX(order_date) FROM orders o2 WHERE o1.customer_id =
o2.customer_id);
```

WHAT THIS QUERY DOES

The correlated-subquery pattern applied to `orders` instead of `employees`: for each order, check whether its date matches the *most recent* date among all orders placed by that same customer. Rows that pass are exactly each customer's single latest order.

RESULT ON OUR SAMPLE DATA

customer_id	order_date	amount
4	2024-02-10	32,000
5	2024-02-15	45,000
6	2024-03-01	21,000
7	2024-03-05	67,000
8	2024-03-10	34,000
9	2024-03-15	28,000

customer_id	order_date	amount
10	2024-04-01	42,000
1	2024-04-05	15,000
2	2024-04-10	33,000
3	2024-04-15	29,000

10 rows returned.

Q130. Oldest order for each customer MEDIUM

```
SELECT customer_id, order_date, amount
FROM orders o1
WHERE order_date = (SELECT MIN(order_date) FROM orders o2 WHERE o1.customer_id =
o2.customer_id);
```

WHAT THIS QUERY DOES

The mirror image of Q129, using `MIN` instead of `MAX` to find each customer's very first order instead of their latest.

RESULT ON OUR SAMPLE DATA

customer_id	order_date	amount
1	2024-01-10	15,000
2	2024-01-20	12,000
3	2024-02-01	50,000
4	2024-02-10	32,000
5	2024-02-15	45,000
6	2024-03-01	21,000
7	2024-03-05	67,000
8	2024-03-10	34,000
9	2024-03-15	28,000
10	2024-04-01	42,000

10 rows returned.

Q131. Employees hired after average hire date MEDIUM

```
SELECT name, hire_date FROM employees
WHERE hire_date > (SELECT AVG(hire_date) FROM employees);
```

WHAT THIS QUERY DOES

A worthwhile pitfall to understand rather than a query to copy as-is: `AVG()` is built for numbers, and a date like `'2020-01-15'` is really just text. Different databases handle `AVG()` on a date/text column differently — some will error out, others will silently coerce it into some numeric interpretation that doesn't correspond to any real calendar date. In this book's SQLite engine, it happens to extract a rough numeric value and the resulting comparison doesn't reliably mean 'hired after the true average date' — the result shown below should be treated as illustrative of the *syntax*, not as a reliable answer. To do this correctly, you'd first convert each date to a comparable number of days (e.g. SQLite's `julianday(hire_date)`, or MySQL's `TO_DAYS(hire_date)`), average *that*, and then convert back — a good reminder that not every query that runs without an error is actually computing something meaningful.

RESULT ON OUR SAMPLE DATA

name	hire_date
Aarav Sharma	2020-01-15
Priya Patel	2019-03-20
Neha Gupta	2021-02-01
Ananya Menon	2022-06-15
Meera Krishnan	2020-08-12
Sneha Patel	2023-01-20
Rajesh Kumar	2019-06-10
Deepa Iyer	2020-11-15
Kavya Nair	2021-08-20

9 rows returned.

Q132. Department with most employees HARD

```
SELECT department
FROM employees
GROUP BY department
HAVING COUNT(*) = (SELECT MAX(emp_count)
                   FROM (SELECT COUNT(*) AS emp_count FROM employees GROUP BY
department) AS t);
```

WHAT THIS QUERY DOES

A subquery *inside a subquery*: the innermost query groups employees by department and counts each group's size; the middle layer wraps that as a mini result-table (aliased `t`) and takes the overall maximum count from it; the outer query then keeps whichever department(s) match that maximum. This is essentially 'find the argmax' — the group with the biggest count — done by first computing all

the counts, then finding the biggest one, then matching back to find which group(s) achieved it.

RESULT ON OUR SAMPLE DATA

department
IT

1 row returned.

Q133. Department with highest total salary HARD

```
SELECT department
FROM employees
GROUP BY department
HAVING SUM(salary) = (SELECT MAX(total_salary)
                      FROM (SELECT SUM(salary) AS total_salary FROM employees
GROUP BY department) AS t);
```

WHAT THIS QUERY DOES

The exact same triple-nested 'find the argmax' structure as Q132, just aggregating with `SUM(salary)` instead of `COUNT(*)` — a good example of how the same query *shape* answers 'which group has the most people' and 'which group costs the most in salary' with only the aggregate function swapped.

RESULT ON OUR SAMPLE DATA

department
IT

1 row returned.

Q134. Employees with salary in top 3 MEDIUM

```
SELECT name, salary
FROM employees
WHERE salary IN (SELECT DISTINCT salary FROM employees ORDER BY salary DESC LIMIT
3);
```

WHAT THIS QUERY DOES

The subquery finds the 3 highest distinct salary *values* (not employees — if two people tied for 3rd place, this still only picks 3 salary values), and the outer query then finds every employee earning one of those values. Worth noting this can return *more than 3 employees* if there are salary ties among the top 3 values — a subtly different result than a strict 'top 3 people' query.

RESULT ON OUR SAMPLE DATA

name	salary
Vikram Reddy	90,000
Karthik Nair	95,000
Ravi Teja	91,000

3 rows returned.

Q135. Employees with salary in bottom 3 MEDIUM

```
SELECT name, salary
FROM employees
WHERE salary IN (SELECT DISTINCT salary FROM employees ORDER BY salary ASC LIMIT
3);
```

WHAT THIS QUERY DOES

The mirror image of Q134 — the 3 lowest distinct salary values, and everyone earning one of them.

RESULT ON OUR SAMPLE DATA

name	salary
Ananya Menon	60,000
Sneha Patel	58,000
Deepa Iyer	64,000

3 rows returned.

Q136. Customers with at least 2 orders MEDIUM

```
SELECT name FROM customers
WHERE customer_id IN (SELECT customer_id FROM orders GROUP BY customer_id HAVING
COUNT(*) >= 2);
```

WHAT THIS QUERY DOES

The subquery does its own grouping and filtering internally (GROUP BY + HAVING, from Level 3) to find customer IDs with 2 or more orders, and the outer query then looks up those customers' names. This shows subqueries aren't limited to single-value results — a subquery can itself be a fully grouped, filtered mini-query, as long as its final output is a single column that the outer `IN` can check against.

RESULT ON OUR SAMPLE DATA

name
TechCorp
GlobalSolutions

name
InnovationLabs

3 rows returned.

Q137. Products above average price EASY

```
SELECT product_name, price FROM products
WHERE price > (SELECT AVG(price) FROM products);
```

WHAT THIS QUERY DOES

The same simple 'compare against one aggregate value' subquery from Q114/115, applied to the `products` table's `price` column instead of employee salaries.

RESULT ON OUR SAMPLE DATA

product_name	price
Laptop Pro	75,000
Enterprise Software	100,000
Cloud Storage Plan	50,000

3 rows returned.

Q138. Employees working on active projects MEDIUM

```
SELECT DISTINCT e.name
FROM employees e
WHERE e.department IN (SELECT department FROM projects WHERE status = 'Active');
```

WHAT THIS QUERY DOES

The inner query finds every department currently running at least one Active project; the outer query then lists employees belonging to any of those departments. `DISTINCT` matters here because the inner subquery could list the same department multiple times (once per active project in it), which would otherwise risk showing an employee's name more than once if `IN` were somehow sensitive to duplicate list entries — though in practice `IN` already de-duplicates its comparison list, so DISTINCT here is really about making sure each employee's name appears only once in the final output, not about the subquery itself.

RESULT ON OUR SAMPLE DATA

name
Aarav Sharma
Rahul Singh
Neha Gupta

name
Vikram Reddy
Karthik Nair
Arjun Reddy
Rajesh Kumar
Suresh Babu
Kavya Nair
Ravi Teja

10 rows returned.

Q139. Customers with high value orders EASY

```
SELECT name FROM customers
WHERE customer_id IN (SELECT customer_id FROM orders WHERE amount > 50000);
```

WHAT THIS QUERY DOES

A simple `IN` subquery: find every order over 50,000, collect their customer IDs, and look up which customers those belong to. Note a customer with *multiple* qualifying orders would still only appear once in the final list, since `customers.customer_id` is unique per customer regardless of how many matching order IDs led to it being included.

RESULT ON OUR SAMPLE DATA

name
DeltaCorp

1 row returned.

Q140. Employees with bonus above average EASY

```
SELECT name, bonus FROM employees
WHERE bonus > (SELECT AVG(bonus) FROM employees);
```

WHAT THIS QUERY DOES

The same simple aggregate-subquery comparison from Q114/137, one more time on the bonus column — by now this shape ('filter rows against one number computed by a subquery') should feel completely routine, which is exactly the point of seeing it repeated across several different columns and tables throughout this level.

RESULT ON OUR SAMPLE DATA

name	bonus
Rahul Singh	15,000
Vikram Reddy	12,000
Karthik Nair	20,000
Arjun Reddy	11,000
Suresh Babu	18,000
Ravi Teja	16,000

6 rows returned.